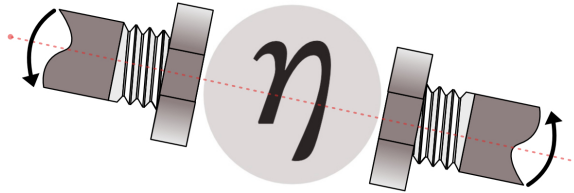


Energy, Volume, and Performance: Post-Moore’s Law Scaling

The Moral Imperative of Energy Efficiency

Andrew N. Sloss
VAIRE COMPUTING
Seattle, Washington State, USA
andrew@vaire.co



Abstract

RELEASED AUGUST 17, 2026: *Buckminster Fuller*, the inventor of the geodesic dome, described a broader philosophy of improvement called *ephemeralization*, often summarized by the famous phrase, “*doing more with less*” [18]. For example, the shift from incandescent light bulbs to LED lights has saved us between ~85–90% on energy [19]. In the history of computation, this idea is reflected most clearly in the semiconductor industry, where silicon geometry scaling has delivered some of the largest gains in energy efficiency over the past 60 years.

With the emergence of *artificial intelligence* and the continued demands to consume more, we seem to have abandoned the efficiency mantra of “*doing more with less*” in favor of the more troubling idea of “*doing more with more*”. Computation has become a race to consume ever-greater resources, where the idea of “*doing more*” remains a belief and a vision rather than a proven benefit. The phrase “*with more*” refers to the use of vast amounts of energy, capital, data, cooling technology, and infrastructure in the pursuit of an uncertain outcome—a modern version of the *mad scientist scenario*: we have achieved our goal but destroyed everything in the process. Our future industrial challenge is to avoid increasing energy to cover up the fundamental fact that we have ended up with “*doing less with more*” technology.

On the other side of the coin, by abandoning energy efficiency at the circuit level, we have come to rely on *suboptimal minimalism*¹. Suboptimal minimalism has ended up being the cornerstone of efficiency, where entire systems are built upon foundations that miss major opportunities. These misses were necessary to get an industry going; today, those same early decisions cause us to struggle to continue advancing technology. We should instead focus on *optimized complexity*². Building circuits that utilize our advanced manufacturing and increased knowledge of physics, rather than being artificially constrained by past simplicity.

Performance at all costs is not sustainable in a long-term race. We need to factor in energy, waste heat, complexity, scalability, and the economics that overshadow everything. Efficiency covers many different areas; in this short article, we look at semiconductor efficiency and the implications of giving up on energy-efficiency for artificial intelligence workloads. Today, the industry focuses on data compression, software architectures, parallel designs, and high speed interconnects to help alleviate the problem, but that will only buy us short-term progress if we do not address the main problem of thermodynamics at the circuit level.

The bottom line is that if we get the energy-efficiency equation wrong, the cost will be more than financial; it may limit the very progress we want to enable. The *moral imperative* is to improve lives, not to consume resources blindly. That requires treating energy efficiency in computation as a foundational design goal, not as an afterthought. We can optimize computation by making operations parallel; we can improve predictive outcomes; we can increase available power; and we can make innovative architectural changes and improve software usage.

These are what we can do today, but tomorrow all paths lead to *Volumetric computing (Energy, Volume, and Performance)*, driven by improving signal propagation and relying less on diminishing returns from transistor density. Computation, at its core, is about data movement and energy efficiency [5].

A computation can be *energy efficient* without being *power efficient*. Power efficiency measures how much energy is used at a given moment, while energy efficiency measures the total energy used over time. A system may use high peak power for a short time (burst-computation) and still consume less total energy than a slower system running continuously at a moderate power level for much longer. Energy efficiency and *useful work* are distinct but closely related. The central question comes down to: *how much energy is consumed performing useful work rather than being dissipated as waste heat and added cooling?*

¹Reducing circuits beyond the point where simplicity improves outcomes.

²The minimum complexity required to achieve the best result.

1 Background

WHY SHOULD WE CARE? Computation is as much a foundational technology as agriculture, transportation, or medicine. Without it, modern civilization simply stops; we have come to rely increasingly on computation for everyday life. It provides the boundary on what we can do. The global complexity is the simplest explanation for why the semiconductor industry remains relatively insular compared with other industries. Incremental progress built on legacy design choices (suboptimal minimalism approach) is reaching its limits; the economic and technological costs of continuing down the same path are becoming too great to ignore.

Chasing computational speed without constraint is how you win the battle and lose the war. If the hammer works, then repeat without regard for better methods. Wasted opportunity. In the end, society catches up and demands efficiencies. Choosing energy efficiency as early as possible is cheaper, easier, and more consequential than retrofitting it at a later stage. But our obsession with performance first is likely to cause long-term damage and is highly difficult to reverse.

Because computation is now ubiquitous, energy efficiency³—has become essential. Energy is a fungible resource; it can come from many sources, but wasting it becomes increasingly costly as computation scales. The more energy-efficient we make computation, the further we can scale it, and the more performance we can achieve within the same physical and economic limits. Achieving this usually requires greater system complexity, using increased knowledge to extract more useful work from the same amount of energy.

Inefficient energy use in computation does not merely waste energy; it limits every downstream benefit that computation is meant to enable. As more energy is consumed by computing infrastructure, fewer resources remain available for other equally important priorities. This creates a broader constraint on technological and social progress. In other words, poor energy efficiency in semiconductors is not only a semiconductor problem; it is a system-wide problem with implications across the entire economy.

JEVONS PARADOX: No efficiency discussion is complete without a mention of the *Jevons paradox*. An economist named *William Stanley Jevons* put forward the notion that efficiency and consumption are related in his 1865 book titled *The Coal Question* [10]. Jevons noticed a paradox: *the more efficiently we make a resource available, the cheaper and easier it is to use. Subsequently, the more people end up using it, providing next to no savings.* In some ways, this is what drives the semiconductor industry, with an endless development cycle; we make incremental changes to enhance energy-efficiency and then produce more chips to take advantage of those enhancements. It is a paradox that has driven our industry until quite recently. For all intents and purposes, energy-efficiency has been pushed aside. This is partly due to the advantages of geometric shrinkage dramatically slowing down, which forces the industry to use significantly more resources to achieve computational performance gains.

³The standard symbol for efficiency in physics, engineering, and thermodynamics is the lowercase Greek letter η (eta)

NASH EQUILIBRIUM: The *Nash equilibrium* is named after the economist *John Nash* [17], although related ideas can be traced back to earlier models of competition among sellers. In our case, the “sellers” are the actors in the semiconductor industry. For more than 60 years, designers, foundries, EDA vendors, materials suppliers, packaging companies, and customers have optimized around the highly successful planar silicon model. As a result, each actor has strong incentives to remain within the existing CMOS ecosystem, where improvements have historically come from geometric scaling [17]. This creates a *Nash equilibrium*: staying on the familiar path makes sense, even if the physical walls become increasingly visible.

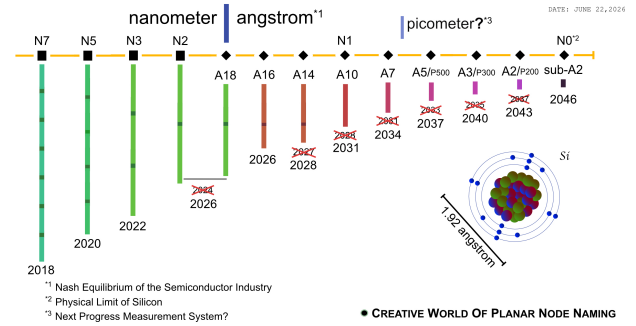


Figure 1: Planar Geometry scaling [9]

ZENO’S DICHOTOMY PARADOX: *Zeno’s Dichotomy Paradox* states that to reach a destination, one must first travel half the distance, then half of the remaining distance, and so on, seemingly never arriving [23]. A similar effect appears in semiconductor scaling. Each new process node promises a move toward smaller geometries, yet the measurement system itself keeps changing; see Figure 1. As a result, the industry appears to be advancing toward ever-smaller dimensions, while the meaning of those dimensions becomes increasingly unclear to define.

WHY IS ENERGY EFFICIENCY A MORAL IMPERATIVE? *Why do we use the word imperative when describing energy efficiency?* Most people would agree that efficiency is important and may carry ethical implications, but they may not see it as something that should override all other priorities. There are many other moral demands: helping the less fortunate, improving food production, decarbonizing electricity, and expanding access to education and healthcare. In many cases, these goals may appear to have higher leverage per unit of effort.

However, the growing scale of computation changes the nature of the question. As computation becomes embedded in nearly every part of modern life, inefficient computing no longer represents just a technical weakness; it becomes a system-level cost imposed on society. This link between energy efficiency and moral responsibility is not new. Historical examples, such as the London smog of the nineteenth and twentieth centuries [2], show that energy choices can produce social consequences that are not morally neutral.

For this reason, energy efficiency in computation should be understood as a moral imperative. The issue is not merely whether we

can build more powerful systems, but whether we can do so without wasting energy, increasing pollution, or diverting resources from other important human needs. The imperative is therefore to shift digital-circuit design from *suboptimal minimalism* toward *optimal complexity*: enough complexity to reduce waste, improve efficiency, and move computation to the next level of capability.

2 the Axioms : Version 5

WHY SHOULD WE CARE? Modern computing systems are governed by a set of industrial axioms. These are the foundational assumptions that cannot be avoided without a substantial reinvention of the computing industry. For better or worse, we live in a digital context. The quantum world may be waiting for us, but for now, most of us are still firmly stuck counting in ones and zeros.

Ax1: **COMMUNICATION** occurs in a digital and asynchronous form; ultimately, the results must be represented digitally, if we want the devices to talk to the outside world.

Ax2: **MAIN MEMORY** is digital, typically implemented as SRAM, DRAM, or HBM. There are overheads in conversion and movement, some can be mitigated with clever buffers (including providing local analog memory) etc. but the digital memory is still the main method of storage, meaning there is a fixed penalty cost in storing and moving information.

Ax3: **ORCHESTRATION**, *control*, and decision-making are performed through digital computation, using software and a Turing Complete Machine (e.g., CPU, GPU, or DSP). Noting that the accelerators can be Turing Incomplete, and it is the orchestration that makes them complete. Even in artificial intelligence workloads, classical computing remains up to 50% of the overall computation, and upto 90% of latency in agentic environments. Classical computing orchestrates data flow, data marshalling, memory management, and even running the IDE tools.

Ax4: **PERFORMANCE** is a careful balance of memory throughput (local), communication bandwidth (global), and compute speed (local). Making the compute too greedy causes the I/O bus to saturate on writes and starves the accelerator on reads; insufficient compute leaves the pipeline underutilized. Peak efficiency requires a co-designed approach rather than independently optimized. These points are sometimes hard to comprehend, “*why does my laptop not run faster using a faster processor?*” it is all to do with the *system* effects. Those effects extend up into software.

Ax5: **THE SILICON ADVANTAGE** relies on *continuous computation*, *economies of scale*, *repeatability*, and *adaptability* [19]. Historically these attributes have pushed the industry from narrow specialization toward general-purpose platforms. Some applications, however, create a new form of generalization. Artificial intelligence may be one example: mathematically specialized, but broad enough in use to become a new kind

of general-purpose computing.

Ax6: **COMPLEXITY** has a cost. That cost may appear as longer development cycles, greater verification burden, lower yield, larger physical area ($area \equiv cost$), or increased integration risk. There is no free lunch. However, when one system attribute falls out of alignment, the design focus must shift toward restoring balance. Complexity is justified only when realigns is required. Today, energy efficiency is out of alignment.

Ax7: **POWER EFFICIENCY** is the measurement of power consumption in a moment of time. Often used to identify peak and minimal power requirements, see Appendix B for details.

Ax8: **ENERGY EFFICIENCY** is the integral of power consumption over time. Often used to provide the average energy characteristics of a device, see Appendix B for detail.

Ax9: **USEFUL WORK** is the amount of energy that is actually transformed in to useful work ($E_{in} - E_{waste}$), see Appendix B for details.

Ax10: **TODAY**, computation remains overwhelmingly digital. Non-digital techniques are possible, but these are mostly limited to specialized domains such as power delivery, i/o interfacing, sensing, and oscillation. Programmable digital logic provides the vast majority of the computation. Adaptable through software, allowing the solutions to change with the problem.

In summary, any computational system must operate within the constraints of digital logic, whether the system is photonic, analog, quantum, or biologically based. Digital logic controls communication, memory, orchestration, and the software interfaces. It is the glue we use to combine everything together.

3 Methods of energy-efficiency

WHY SHOULD WE CARE? We should always examine the popular methods of today. These methods improve energy efficiency in circuits, ranging from usage patterns to implementation techniques. For a deeper discussion, see Appendix A for the economics equations and Appendix B for the power and energy efficiency equations. Computation can be understood as a sequence of discrete steps that, when carefully ordered and executed at scale, resemble a continuous transformation [19].

A useful way to view computation, efficiency, and production scale is to see computation as an abstract sequence of operations in an industrial context. Using computation as the subject, we can paraphrase Brian Potter on efficiency: “*continuous computation does not merely enable large-scale production; rather, large-scale production is often a precondition for making continuous computation viable. The reason is that the supporting EDA tools, semiconductor infrastructure, verification effort, and manufacturing complexity required to sequence each step can usually be justified only when amortized across massive production volumes*” [19].

This links energy efficiency, computation, and production volume together. Efficiency is not only a property of the circuit itself; it is also shaped by the scale at which the circuit can be designed, manufactured, verified, and deployed.

Below is a subset of energy-efficiency methods used today:

- Ee1: **RACE-TO-HALT:** This method has conceptual roots in the *Turing Machine*: complete the computation, produce the result, and then stop. Since

$$\text{Energy} = \text{Power} \times \text{Time}$$

reducing the time component can reduce total energy consumption, even if the system uses high peak power during execution. The goal is not necessarily to minimize instantaneous power, but to minimize the total energy required to complete the task. This strategy assumes that idle power is significantly lower than active power. Many modern electronic systems use this approach by increasing frequency and voltage for short burst periods, completing the workload quickly, and then returning to a low-power idle or sleep state. However, race-to-halt is not always optimal. A chip with excellent peak performance but poor idle efficiency may perform badly on bursty workloads if it spends too much energy while waiting. Conversely, a chip with only moderate peak performance but excellent idle efficiency may achieve better total energy efficiency, especially in battery-powered systems, even if its sustained *performance per watt* appears lower.

- Ee2: **DYNAMIC POWER SCALES:** Voltage is the leverage everything pivot around with dynamic power:

$$P_{\text{dynamic}} = \alpha \cdot C \cdot V_{DD}^2 \cdot f$$

This is easily seen with the V^2 variable. By reducing voltage by 20% dynamic power can reduce by ~36%. BUT, we are also reducing frequency that causes the workload to take longer to complete, remember we are dealing with energy, not power. What is interesting if we cut both V and f together, the power drops *quadratically* in V while time increases roughly linearly in $1/f$. This means energy for any particular workload is dominated by V^2 , and not by f . That is the main reason we see *Dynamic Voltage and Frequency Scaling* (DVFS). There are points on the curve which are super energy efficient for a implementation, while staying above a voltage threshold.

- Ee3: **RULES OF THUMB:** We can also view power through the close relationship between voltage (V) and frequency (f). However, higher operating frequencies often require higher supply voltages. This acts as a conditional. Under the simplifying assumption that voltage scales roughly with frequency, this leads to the useful *rule-of-thumb* (or *Geiger observation*) :

$$P_{\text{dynamic}} \propto f^3$$

This means that increasing frequency can cause power to rise approximately cubically, not merely linearly. Frequency scaling therefore changes more than performance; it also changes the voltage requirement, thermal profile, and energy behavior

of the circuit.

- Ee4: **SUBTHRESHOLD:** There is a minimum-energy threshold that should not be confused with the minimum-power point. If the supply voltage is pushed too low, instantaneous power may continue to fall, but execution time increases. As the workload takes longer to complete, leakage power remains present and can become a dominant part of the total energy cost. The total energy for a workload can therefore be viewed as the sum of dynamic energy and leakage energy:

$$E_{\text{total}} = E_{\text{dynamics}} + (P_{\text{leakage}} \times \text{time})$$

Where E_{dynamic} is the energy used for switching activity, P_{leakage} is the leakage power, and (t) is the time required to complete the workload. There is a U-shaped curve with a minimum somewhere around $0.3 - 0.4V$ for modern CMOS processes. This is what is called the *subthreshold* pioneered by *Anantha Chandrakasan* and others [3]. Optimizing for power alone drives you off the bottom of the U; optimizing for energy we avoid *suboptimal minimalism* of the circuit.

- Ee5: **DENNARD SCALING:** The end of Dennard scaling changed the semiconductor industry. Until roughly the mid-2000s, geometric scaling reduced power per transistor, allowing more computation without a proportional increase in power density [4]. For a time, higher density, better performance, and improved energy behavior advanced together. We can also call it the point when single thread processors stopped providing good returns, and the focus changed to multi-core designs (based on *symmetric multiprocessor* and *symmetric multi-threaded* designs) [28].

When Dennard scaling weakened, the alignment between transistor density and energy efficiency broke. More transistors could still be placed on a chip, but they could not all be powered simultaneously within a practical thermal budget. This gave rise to *dark silicon*: transistors or specialized blocks that would remain inactive until needed because the full chip could not operate within the allocated power budget [6].

Although this may look wasteful from an area perspective, it can improve energy efficiency. By activating only the specialized blocks needed for a workload, the system can reduce unnecessary switching and complete tasks with lower total energy. This shift laid the groundwork for *Domain-Specific Architectures*: chip architectures optimized for particular classes of workloads, such as *machine learning*, rather than for general-purpose computation [8].

- Ee6: **COMPRESSION:** The next method of energy efficiency is about compressing the data. For artificial intelligence problems, we see organization playing with datatypes compression (to *Int4*, *FP4*, and *BF16*). The compression is in the hope that it reduces data storage requirements and cost of data movement for models. Datatypes effect energy efficiency, the smaller we make the data being moved around the more energy efficient the system, but with compression comes at a cost, loss in precise and sometimes accuracy, also there is a rate of diminishing

returns. The energy saving can only be captured once. However, compression may also make it possible to build pattern recognizers that avoid unnecessary computation. For known inputs, the system can return a standard result directly, rather than performing the full computation again, thereby reducing energy use.

Ee7: **INTERCONNECTS:** How computational components are connected is fundamental to energy efficiency. Greater *composability* allows a system to configure communication paths, memory access, and compute resources around the needs of a workload. Biological neurons provide an extreme example: their flexible connectivity allows information to flow through many possible paths rather than through a fixed pipeline. In engineered systems, technologies such as optical interconnects can improve long-distance communication within data centers, but the larger goal is system-level flexibility. High composability allows software to place work where communication cost is lowest, reducing data movement and improving total energy efficiency.

Ee8: **HETEROGENEOUS ARCHITECTURES:** Heterogeneous architectures arose because the workloads required a broader energy-efficiency profile beyond a single pipeline architecture. The challenge was to provide different profiles while the software executes on both. The complete heterogeneous architectures can cover general-purpose CPUs with accelerators (GPUs, FPGAs, ASICs). Taking general-purpose CPUs as an example, there is heterogeneity in the pipelines. The multi-core design could include $Core_{perf}$ (with *Out-Of-Order* pipelines) and $Core_{eff}$ (with *In-Order* pipelines). Allowing a system to tune for either high-performance peak throughput or energy efficiency sustained low-power operations. Complexity shifts to the software scheduler, which must determine the best use of each core type depending on the workload, performance target, and available energy. This also allows for graceful degradation as the battery or energy source becomes depleted, reducing performance in a controlled way rather than failing abruptly.

Ee9: **THE RED HERRING:** Thermal Design Power (normally called TDP) is a power specification and obviously not energy. TDP gives us a important hint on the required peak cooling requirement. Data-center procurement that optimizes on TDP alone risks ignoring the full duty cycle of the workload, including active time, idle time, utilization, and workload variability. As a result, it may lead to the wrong silicon decision: selecting a device that looks efficient by peak power but performs poorly in total energy consumed. Energy efficiency involves looking at the complete workload cycle.

Ee10: **ALTERNATIVES:** There are hybrid alternatives for improving energy efficiency, including *analog computing* and *optical communication*. As discussed in the *Axioms* Section 2, however, these technologies must still co-exist with the digital world. Photonics is extremely fast for point-to-point communication, especially over longer distances (useful in datacenter

scenarios). There is a cost of converting electrons-to-photons and photons back to electrons. These conversions add latency, complexity, and thermal overhead. Efficiency gains must be weighed against the overhead of auxiliary functions.

Similarly, analog computing can be highly effective for certain workloads, but it faces comparable conversion costs when data moves between digital memory and analog computation. It also introduces additional challenges in programmability, precision, repeatability, and adaptability. There are gains in energy efficiency, but it comes at a high cost of non-adaptability. These technologies can improve energy efficiency in the right context, but they are not free replacements for digital computation. Each alternative shifts the constraint rather than removing it.

Ee11: **MEASUREMENT:** To create a fair method of comparison between systems, we have use a standard benchmark. There are two styles, Performance-per-watt for work, measured at a particular operating point. For useful work we measurement *performance-per-joule*, or its workload-specific variants such as *joules-per-inference*, *joules-per-token*, or *joules-per-frame*. Specifically for *Large Language Models* we have *joules-per-token*. This last metric can be used to determine electricity bills, and accelerator choice. Performance-per-watt is misleading since it does not account for burst, idle, and memory system overheads.

4 the Workloads

WHY SHOULD WE CARE? Comparing energy efficiency without context is like judging a car only by its color and wheels. The context determines the operating constraints, and not all computations are the same; some produce a single real-time response, while others produce continuous responses.

Energy efficiency is directly related to the workload and the execution engine. The workloads can be embarrassingly structured, embarrassingly parallel, embarrassingly compressible, and embarrassingly unhelpful. Computational engines can vary between specialization and general purpose. Specialization gives us greater energy efficiency but less flexibility, whereas general-purpose systems are less energy efficient but provide much greater flexibility. Specialization favors structured, parallel, and compressible workloads, where the problem can be mapped efficiently onto dedicated hardware. General-purpose processors are better suited to irregular, unpredictable, or poorly structured workloads that require flexibility. A general-purpose engine can usually execute specialized workloads, although not always efficiently. A specialized engine, however, does not necessarily handle general-purpose workloads at all. It is worth noting that specialized compute engines lean towards more mature and stable solutions due to their limited flexibility.

HEAVY WORKLOADS: For workloads that require sustained-throughput (commonly known as heavy workloads), performance-per-watt at any point will closely approximate joules-per-task, making the efficiency approximately equal, i.e., $E_{power} \approx E_{energy}$. To make it energy efficient, we concentrate on an operating point, not

necessarily the workload.

BURSTY WORKLOADS: It is the opposite; performance-per-watt does not represent energy efficiency. For energy-per-task, we have to use a more realistic scenario and potentially measure per sample. Performance-per-watt provides a misleading measurement.

BATTERY DRIVEN WORKLOADS: For battery-constrained devices, the working characteristic should always be a race-to-halt with deep idle, hoping that deep idle provides the majority of the time. This requires clock⁴ and power⁵ gating. Additionally, retention SRAM and fast wake-up technology are important. All these attributes may affect overall performance-per-watt.

DATACENTER WORKLOADS: The electric bill, joules-per-query (or per-token, per-transaction), is the only number that matters. Two chips with identical TDP can differ by multiples in joules-per-computation, depending on how well an architecture matches the workloads being executed.

HIGH PERFORMANCE COMPUTING WORKLOADS: High performance computers face a struggle between the workloads that demand the most compute, which generate the most heat, and the cooling of that heat, which consumes a significant fraction of the total facility energy. Cooling typically covers between 40% and 60% of the energy budget [24]. Modern HPC energy efficiency is measured in gigaflops-per-watt, with gains coming primarily from heterogeneous architectures that pair general-purpose CPUs with accelerators.

SPACE-BASED WORKLOADS: *Space-based datacenter* workloads require new architectures to handle the hostile environments of space [26]. These architectures have to be radiation tolerant and energy efficient. Lower energy efficiency means more support hardware is required to be placed in orbit; every watt of waste heat increases the burden on radiators, power systems, and launch mass. For these applications, energy efficiency directly involves thermal understanding. Energy efficiency is directly linked to mass; the higher the mass, the more expensive the solution.

Difficulties

The idea of perfect workloads also has important limitations. *James Hamilton, Senior Vice President and Distinguished Engineer at Amazon*, kindly provided three important counterarguments as an example:

Jh1: **LIMITED HORIZONTAL SCALING FOR SOME WORKLOADS:** Not all workloads scale efficiently across an increasing number of servers. Machine Learning, in particular large-scale training and some inference workloads, often exhibits diminishing returns as cluster size grows because of communication,

synchronization, and memory bottlenecks. In these cases, vertical (scale-up) performance remains an essential requirement for increasing computational throughput.

Jh2: **POWER IS ONLY ONE COMPONENT OF TOTAL COST OF OWNERSHIP:** Although reducing energy consumption is desirable, electricity is often a relatively small fraction of the total cost of operating high-end accelerators such as GPUs. Hardware acquisition, depreciation, networking, cooling, and facility costs typically dominate. Consequently, customers may prefer to consume more power if doing so increases utilization of these expensive computing resources and improves return on investment.

Jh3: **PERFORMANCE OFTEN OUTWEIGHS ENERGY EFFICIENCY:** In many commercial and scientific workloads, computation speed has greater economic value than minimizing power consumption. Customers are frequently willing to pay a premium for faster execution because it reduces time-to-result and increases overall productivity. For example, SRAM-only systems or conventional processors could achieve substantially lower power consumption by operating at reduced clock frequencies, but this would come at the expense of performance. In practice, higher performance often provides greater value than lower energy consumption alone.

4.1 Summary

In summary, power efficiency tells us about the silicon, while energy efficiency tells us about the full solution: silicon, workload behavior, idle time, secondary activity, and software integration. Secondary activity includes shared resources and background tasks that continue to consume energy beyond the primary workload. Energy efficiency, therefore, captures the real-world overheads that simple power measurements can miss. As *James Hamilton* points out, the full picture is more complex, but the analysis must still begin at the semiconductor level. Semiconductors set the foundation for efficiency; if the devices are inefficient, the rest of the stack can only compensate, not solve the problem. Those inefficiencies propagate upward into power delivery, cooling, scheduling, and software integration. When performance becomes the only goal, the entire stack is forced to trade efficiency for speed.

5 Working laws and principles

WHY SHOULD WE CARE? The semiconductor industry is shaped by a set of economic observations and engineering principles that help predict the direction, pace, and limits of technological progress. Listed here are a subset of the more important rules. We will start with Wright's Law.

5.1 Wright's Law, 1936

Wright's Law, often described as the learning curve or experience curve, is a principle stating that each doubling of cumulative production reduces production costs by a roughly constant percentage.

⁴clock gating, which saves dynamic power by stopping the clock signal from reaching parts of a circuit that are not currently required,

⁵power gating, which saves leakage (static) power by completely disconnecting power from unused circuit blocks.

In computation, the same broad idea can be applied to learning-driven efficiency: as systems are designed, manufactured, deployed, and refined at scale, we discover better ways to build and operate them. These improvements may come from process refinement, architectural optimization, better tools, or even transitions to entirely new methods of computation [19].

5.2 Landauer’s Principle, 1961

Landauer’s principle states that erasing information has a minimum unavoidable energy cost (culminating in waste heat) [12]. Per bit erased:

$$E_{\min} = k_B T \ln 2 \quad (1)$$

Where:

- k_B = Boltzmann’s constant
- T = temperature in kelvin
- $\ln 2$ = natural logarithm of 2

Related to energy efficiency, it means there is a physical lower bound on how efficient conventional irreversible computation can become. As devices become more efficient, they move closer to this limit, and further improvements become harder.

5.3 Moore’s Law, 1965

In some ways, Moore’s Law is a version of Wright’s Law. Moore’s Law is the observation that the number of transistors on a microchip doubles roughly every two years [16], due to increased knowledge and skill, while the cost of computers is cut in half. The observation, with a few tweaks, has remained true for the semiconductor industry for the past 60 years. But we are getting to the physical limits. A silicon atom is just under 2 angstroms in width, and we are building systems that claim to be 10 angstroms wide. Planar geometry shrinkage will come to an end either for economic reasons or when we hit the physical limits.

COMPARISON: Comparing Moore’s law with Wright’s law. As pointed out by the Professor of *Complex Systems* at the *University of Oxford*, *J. Doyne Farmer*, Moore’s law implies that costs will go down no matter what happens with the technology [7]. By comparison, Wright’s law implies that costs will only go down if effort is applied. The big questions are: *Do costs go down as production goes up*, or *Do reduced costs cause production to increase*? According to Farmer, looking at specific data, Wright’s law appears to be a better and more general representation of technological progress, so increasing production reduces costs.

5.4 Amdahl’s Law, 1967

Amdahl’s law is related to the law of diminishing returns. The law of diminishing returns presents the concept that, beyond a certain point, increasing complexity or resources produces progressively smaller improvements in output, performance, or efficiency. Amdahl’s Law can be expressed as

$$S = \frac{1}{(1 - P) + \frac{P}{N}} \quad (2)$$

Where:

- S = the total speedup
- P = the parallelizable fraction of the workload
- N = the number of processors or the improvement factor

Amdahl’s Law shows that system performance is constrained by the least scalable part of the workload [1]. For electrical systems, that part is the serial part of a system. Amdahl’s Law shows that total energy savings are limited by the parts of the system that remain inefficient, such as memory movement, interconnects, cooling, power delivery, or serial control logic.

5.5 Koomey’s law, 2010

Koomey’s Law directly focuses on energy efficiency [11]. It is the observation that computations per joule have been doubling roughly every 1.5–2.5 years for decades, even as Dennard scaling ceased and clock-frequency scaling stalled. *What does this mean?*

It means that computers have been getting more energy-efficient over time, but, and it is an important "but", the old easy efficiency curve is slowing, and future progress requires seriously different thinking to keep scaling advancing [20]. In other words, it leads to the conclusion of optimal complexity, as mentioned at the start of this paper. It is also worth noting that peak power per chip actually increased over the same period. It is the work-per-joule that has improved.

5.6 Tau (τ) Scaling Law, 2026

An interesting recent addition to the discussion of semiconductor scaling laws is the *Tau (τ) Scaling Law* (sometimes referred to as the *Her Law*), introduced by *Huawei* [31]. It argues that future performance gains increasingly depend on reducing the signal propagation time constant (τ) across devices, circuits, and systems. In physical terms, this means shortening communication distances, lowering parasitic resistance and capacitance, and organizing logic more efficiently in space. As an example implementation, Huawei introduced a three dimensional concept called *logic folding* [22] that reduced the physical distances.

The core idea is that both performance and energy efficiency improve when signals travel shorter paths through better physical organization. This aligns with the broader shift away from purely planar design toward volumetric, three-dimensional integration. By using the vertical dimension, systems can place memory, logic, interconnects, and power delivery closer together, reducing delay and energy loss while enabling greater architectural flexibility.

This suggests that future scaling may depend less on shrinking individual transistors alone and more on reducing the physical distance that information and energy must travel through the interconnect architectures.

5.7 the Law Argument

There is an argument between the backers of the traditional Moore’s Law scaling and the supporters of the newer Tau (τ) Scaling Law [30]. Moore’s Law is historically based on the increasing density of transistors over time, while the Tau (τ) Scaling Law emphasizes the reduction of signal propagation time across devices, circuits, and systems. In simplified terms, Moore’s Law continues to improve through transistor scaling, geometric shrinkage, and higher

device density. The argument for the Tau (τ) Scaling Law is that, as miniaturization becomes increasingly difficult and expensive, future gains must come from reducing signal distance, lowering parasitics, and organizing computation more efficiently in physical space.

These two arguments are not necessarily contradictory. They emphasize different bottlenecks in the evolution of computing systems. Moore’s Law focuses on the transistor as the primary unit of progress, while Tau (τ) Scaling focuses on the time and energy costs of moving signals through the system. One law represents an economic observation, and the other represents physics. In current systems, both remain important: better transistor density still matters, but so do shorter interconnects, closer memory placement, advanced packaging, and three-dimensional integration.

The deeper debate is therefore not whether transistor scaling or Tau (τ) Scaling is correct, but which one will dominate future improvements in performance and energy efficiency. As the cost and difficulty of further miniaturization increase, volumetric design becomes increasingly attractive. Moving from planar layouts toward three-dimensional integration allows logic, memory, interconnects, and power delivery to be placed closer together, reducing delay and energy loss. In this sense, the central question is no longer whether computing systems will become more volumetric, but how quickly the transition will occur and in what form.

6 Post Moore’s law future

WHY SHOULD WE CARE? The semiconductor industry is locked in a competitive race, and the comfortable path risks becoming a dead end. Today’s equilibrium is stable, but it could become unstable overnight. As a whole, the semiconductor industry is a complicated global ecosystem with material providers, designers, manufacturers, EDA tool providers, and customers. In other words, the equilibrium consists of groups of vested interests that are already aligned around optimizing the existing planar path. *BUT* when nation states are denied access to key technologies along the path, they naturally rewrite the rules and explore different directions, breaking free from the suboptimal minimalism of the current treadmill [15, 31] (see Section 5).

The future of computing will be shaped by two forces: economics and the demand for greater computational efficiency. Neglecting either risks a future that is economically and technologically unsustainable. Significant gains in energy efficiency can only come from greater complexity and innovation at the circuit level—new and different Physics must be brought into the circuit world. To overcome hard problems, we must make things more intelligent or complex [13]. As *James Ang*, Chief Scientist for Computing in the Physical and Computational Sciences Directorate at Pacific Northwest National Laboratory, put it, “*we need a break from the Nash equilibrium and create hundreds of experiments to see what will work for the future*”. It is time to put serious effort into creating new experiments that push the boundaries.

Realizing these advances will require a fundamental shift in how computation is executed. The path is not easy, but it is the path to sustained progress. The technologies discussed here sit at the lower

end of the *Technology Readiness Level* (TRL) scale [14]: they have emerged from basic research but still require commercial validation.

The future path toward volumetric computing must confront the thermodynamic problem that three-dimensional integration creates. Before computation can move deeply into volumetric designs, the system must be able to remove heat from those volumetric structures. Noting that heat is an issue with most computational solutions, i.e., *near-memory compute*, *HBM improvements*, *optical interconnects*, *chiplets*, *backside power delivery*, and many other new areas of development. There are two broad ways to address the thermal problem: first, by mitigating the thermal constraint, and second, by redesigning the circuits so that less waste heat is produced in the first place.

The first approach requires better cooling methods capable of reaching the interior of a three-dimensional silicon structure. For example, microfluidic cooling could use ultra-pure fluids flowing through tiny channels that extend deep into the chip, carrying heat away from regions that conventional surface cooling cannot easily reach [29]. We are not sure of the implications for reliability and effectiveness that this technology could bring to 3D silicon chips.

The second approach is more fundamental: redesign the logic circuits themselves to reduce waste heat at its source. Rather than only improving the removal of heat after it is created, this path asks whether computation can be performed in ways that generate less irreversible energy loss in the first place.

Note: In the *Evolution of the semiconductor industry and the Start of X Law*, a paper written in 2019 and published in 2022, the mythical *X Law* was proposed as a successor to Moore’s Law [27]. Upon further consideration, however, *the Tau (τ) Scaling Law* may be a good foundation for a future law since it is moving in a direction that can still expand even when the current planar limits are reached.

6.1 Adiabatic Reversible CMOS

Reversible computing is the idea that you can design logic so that no information is ever destroyed and, therefore, no energy must be lost as heat; see Landauer’s Principle described in Section 5.2. In adiabatic reversible CMOS, or simply ARC, signals are moved using slow, carefully shaped power-clocks that recycle charge instead of dumping it to the ground rails; see Appendix C for the efficiency equations. Rather than abruptly charging or discharging a node (which wastes $\frac{1}{2}CV^2$), adiabatic logic *pulls* and *returns* energy smoothly, so most of it flows back into the power-clock. In practice, some energy is always lost through resistance, leakage, non-ideal waveforms, and imperfect devices, but the core principle still holds: by avoiding information loss and sharp switching, these circuits can operate with dramatically lower energy requirements and dissipated heat than conventional solutions. Bypassing the theoretical limits of irreversible computing by combining thermodynamic knowledge into circuit design. ARC, as described, has practical barriers (clock overhead, slower operation, EDA toolchain immaturity). These are true problems, but they are not insurmountable problems that money cannot fix—if we care about energy efficiency.

SECONDARY EFFECT: Energy is not only the price of computation; it is also one of its fingerprints. As compute density and

performance increase, heat becomes a visible trace of what the system is executing. Adiabatic Reversible CMOS matters because reducing irreversible energy loss may reduce not only power consumption and cooling demand, but also the thermal side channels created by computation.

6.2 Volumetric Computing

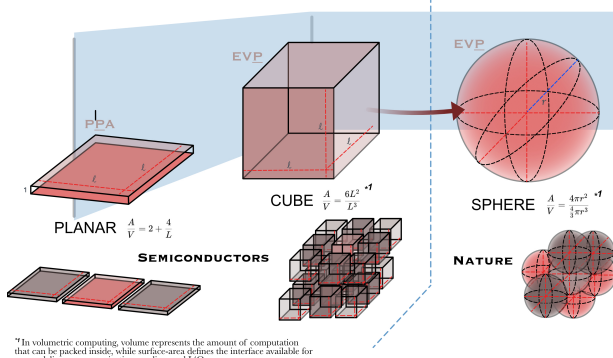


Figure 2: Energy, Volume, and Performance, EVP

By first controlling the thermodynamic factors, we can start solving the 3D interconnect problem. In volumetric computing, volume represents the amount of computation that can be packed inside, while surface-area defines the interface available for power delivery, communication, cooling, and I/O, see Figure 3. For the last 60 years, we have mostly focused on planar implementations. This has been very successful; our progression has continued with geometric shrinkage. Today, we can see that geometric shrinkage is coming to an end, so we need to make the difficult decision to transition to volumetric designs. This puts pressure on both EDA tools and manufacturing, but the benefits include reduced signal distance, as hinted at by *Tau (τ) Scaling Law*; see Section 5.6. We shift our focus from *Power, Performance, and Area*⁶ (PPA), to *Energy, Volume, and Performance* (EVP), where energy efficiency is linked to the workloads and the complete solutions.

A recent IBM announcement offers an early sign that volumetric computing is beginning to emerge. IBM described a 0.7 nm (or A7) process [21] that stacks transistors in two layers using a structure referred to as *Nanostack*. This is not full volumetric computing, but it is an important move beyond purely planar scaling. However, the process does not fundamentally solve the thermodynamic challenge of heat generated by circuits; it still depends on making devices smaller and denser. IBM nevertheless claims that the approach improves both performance and energy efficiency.

6.3 Quantum Computing

We would be remiss not to mention *Quantum computing* in the context of energy efficiency. Quantum computing uses quantum states, such as superposition and entanglement, to process certain kinds of problems differently from classical digital computers. Instead of representing information only as bits (0) or (1), quantum

⁶PPA serves as a reminder that *area* is both a physical quantity and a cost function

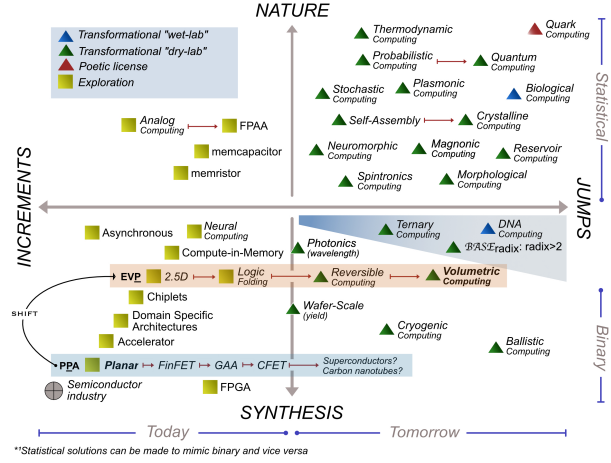


Figure 3: Future of computation engines [25]

computers use qubits, which can exist in combinations of states until measured. This allows quantum algorithms to explore some mathematical structures more efficiently than classical methods.

With respect to energy efficiency, quantum computing could be important if it solves specific problems using far fewer computational steps than a classical machine would require (the energy efficiency time is probably in years compared to a classical computer). In that case, the total energy used to reach a solution could be lower, even if the quantum hardware itself requires complex control systems and, in many cases, cryogenic cooling. However, quantum computing is not a general replacement for standard digital circuits. Most everyday computation, control logic, memory, communication, operating systems, and conventional workloads will continue to rely on classical digital hardware. Quantum computing is better understood as a specialized accelerator for selected problem classes, not as a wholesale replacement for the digital computing stack. Quantum computing has the potential to cut years off classical computing time for a narrow set of problems, and in that context, it could achieve superior energy efficiency.

7 Conclusion

Improving *computation-per-joule* (useful work) is critical if we want civilization to move forward. It is up to the next generation to make energy efficiency a moral imperative. We can see historically that every big jump in semiconductor capability has come from improvements in energy efficiency, and it is energy efficiency that has driven performance, not the other way around. It is the economic argument. Today, we are faced with our biggest challenge: the runway of planar progress is coming to an end. Whether the end is now or in 5 years time, it has ended. It does not matter how well we architect a design; the distances between points are getting longer, and with that come delays in signal propagation in a plane.

We have circumvented each challenge by applying new knowledge and changing the rules of design. For example, the End of Denard's scaling brought about the multi-core/many-core paradigms,

adding heterogeneous cores (with different energy profiles), specialized accelerators, aggressive sleep states, and near-threshold voltage research. These have been amazing technological jumps, but we are facing a new challenge that our planar bag of tricks cannot solve: increasing performance while simultaneously increasing energy efficiency. The next natural progression is likely to be through volumetric computing. To do this, we must shift our focus to the challenges introduced by three-dimensional architectures, particularly their thermodynamic constraints. This gives us a roadmap for shorter distances for signal propagation and denser chip designs by emphasizing volume over planar.

The only method to achieve volumetric computing is to control the thermodynamic characteristics of circuits. Irreversible computing (open thermodynamic circuits) will remain expensive because it generates significant amounts of waste heat per calculation. That heat has to be cooled using additional technology, adding to the cost of computation. It seems that the future must involve closed thermodynamic circuits that reduce waste heat from computation as much as possible. By doing so, we can enter an exciting new era of volumetric computing. Extending Moore’s Law not through planar development, but through 3-dimensional development. *Moore’s law becomes volumetric.*

Our journey has taken us through the concepts of *suboptimal minimalism* $\rightarrow$ *optimized complexity* $\rightarrow$ *thermodynamics* at the circuit level $\rightarrow$ *Energy, Volume, and Performance* (EVP) $\rightarrow$ *useful work*. The journey is a realization that change is expensive, but the lack of change is more expensive. Breaking free from the Nash equilibrium is going to be difficult. The *moral imperative* for the next generation is to increase *energy efficiency*; otherwise, we risk consuming everything in the pursuit of *more*. Energy efficiency is about applying *optimal complexity* to our electronic designs so that more useful work is produced. If we do not do this, we will struggle to reach the next level, and the existing runway will end.

8 Acknowledgments

The paper has been in development for a long time, yet it is still not expressed perfectly. We extend our gratitude to the many exceptional engineers and scientists whose pioneering work makes this project possible. We also thank Rodolfo Rosini, Michael Frank, Hannah Blyton, Avi Geiger, Richard Phelan, William Hooper Rees, James Ang, James Hamilton, and the entire Vaire Computing team for identifying errors and raising insightful questions throughout the editing process.

9 Disclaimer

Written by the authors with limited assistance from artificial intelligence tools. All ideas, arguments, interpretations, and final editorial decisions remain the responsibility of the authors. Tools such as *Grammarly*, *Anthropic Claude*, *Google Gemini*, and *OpenAI ChatGPT* were used to improve grammar, clarity, and style. They were not used to generate the underlying content, facts, arguments, or opinions. The authors remain fully responsible for all content, ideas, interpretations, bad jokes, and conclusions presented here.

References

- [1] Gene M. Amdahl. 1967. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS ’67 Spring)*. ACM, 483–485. doi:10.1145/1465482.1465560
- [2] Michelle L. Bell, Devra L. Davis, and Tony Fletcher. 2004. A Retrospective Assessment of Mortality from the London Smog Episode of 1952: The Role of Influenza and Pollution. *Environmental Health Perspectives* 112, 1 (2004), 6–8. doi:10.1289/ehp.6539
- [3] Benton H. Calhoun and Anantha P. Chandrakasan. 2004. Characterizing and Modeling Minimum Energy Operation for Subthreshold Circuits. In *Proceedings of the 2004 International Symposium on Low Power Electronics and Design (ISLPED ’04)*. ACM, 90–95. doi:10.1145/1013235.1013265
- [4] Robert H. Dennard, Fritz H. Gaensslen, Hwa-Nien Yu, V. Leo Rideout, Ernest Bassous, and Andre R. LeBlanc. 1974. Design of Ion-Implanted MOSFET’s with Very Small Physical Dimensions. *IEEE Journal of Solid-State Circuits* 9, 5 (1974), 256–268. doi:10.1109/JSSC.1974.1050511
- [5] Jack Dongarra, Daniel Reed, and Dennis Gannon. 2026. Ride the Wave, Build the Future: Scientific Computing in an AI World. (2026). <https://netlib.org> Essay exploring the shift of advanced computing towards hyperscale AI platforms and the required evolution of scientific HPC.
- [6] Hadi Esmailzadeh, Emily Blem, Renée St. Amant, Karthikeyan Sankaralingam, and Doug Burger. 2011. Dark Silicon and the End of Multicore Scaling. In *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA ’11)*. ACM, 365–376. doi:10.1145/2000064.2000108
- [7] J. Doynne Farmer. 2024. *Making Sense of Chaos: A Better Economics for a Better World*. Yale University Press, New Haven, CT.
- [8] John L. Hennessy and David A. Patterson. 2019. A New Golden Age for Computer Architecture. *Commun. ACM* 62, 2 (2019), 48–60. doi:10.1145/3282307
- [9] IMEC. 2022. 2D-material based devices in the logic scaling roadmap | imec — imec-int.com. <https://www.imec-int.com/en/articles/introducing-2d-material-based-devices-logic-scaling-roadmap>. [Accessed 24-06-2026].
- [10] William Stanley Jevons. 1865. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines*. Macmillan and Co., London.
- [11] Jonathan G. Koomey, Stephen Berard, Marla Sanchez, and Henry Wong. 2011. Implications of Historical Trends in the Electrical Efficiency of Computing. *IEEE Annals of the History of Computing* 33, 3 (2011), 46–54. doi:10.1109/MAHC.2010.28
- [12] Rolf Landauer. 1961. Irreversibility and Heat Generation in the Computing Process. *IBM Journal of Research and Development* 5, 3 (1961), 183–191. doi:10.1147/rd.53.0183
- [13] James Lovelock. 2019. *Novacene - The Coming Age of Hyperintelligence*. MIT Press. <https://mitpress.mit.edu/books/novacene>
- [14] John C. Mankins. 1995. *Technology Readiness Levels: A White Paper*. Technical Report. Advanced Concepts Office, Office of Space Access and Technology, NASA, Washington, D.C. <https://artemisinnovation.com>
- [15] Chris Miller. 2022. *Chip War: The Fight for the World’s Most Critical Technology*. Scribner, New York, NY.
- [16] Gordon E. Moore. 1965. Cramming More Components onto Integrated Circuits. *Electronics* 38, 8 (April 1965), 114–117.
- [17] John Nash. 1951. Non-Cooperative Games. *Annals of Mathematics* 54, 2 (1951), 286–295. doi:10.2307/1969529
- [18] Alec Nevala-Lee. 2022. *Inventor of the Future: The Visionary Life of Buckminster Fuller*. Dey Street Books / HarperCollins, New York, NY.
- [19] Brian Potter. 2025. *The Origins of Efficiency*. Stripe Press.
- [20] Antonio Prieto, Beatriz Prieto, and Juan J. Escobar. 2025. Evolution of Computing Energy Efficiency: Koomey’s Law Revisited. *Cluster Computing* 28 (2025). doi:10.1007/s10586-024-04767-y
- [21] S. Reboh et al. 2025. NanoStack Transistor Architecture for CMOS 7A Node and Beyond. In *Symposium on VLSI Technology and Circuits*. IEEE/SIA.
- [22] Reuters. 2026. Huawei bets on speed over shrinking transistors to sidestep US chip sanctions. Reuters. <https://www.reuters.com/world/china/huawei-bets-speed-over-shrinking-transistors-sidestep-us-chip-sanctions-2026-05-29/>
- [23] Wesley C. Salmon (Ed.). 2001. *Zeno’s Paradoxes*. Hackett Publishing Company, Indianapolis, IN.
- [24] Md Abu Bakar Siddik, Arman Shehabi, and Landon Marston. 2021. The environmental footprint of data centers in the United States. *Environmental Research Letters* 16, 6 (2021), 064017. doi:10.1088/1748-9326/abfb1
- [25] Andrew Sloss. 2025. Lost in Algorithms: Architecting Complexity. https://www.researchgate.net/publication/367163246_Lost_in_Algorithms_Architecting_Complexity. [Accessed 14-06-2026].
- [26] Andrew N. Sloss. [n. d.]. Unlocking Space-Based Datacenters Through Adiabatic Reversible Computing. https://www.researchgate.net/publication/401355852_Unlocking_Space-Based_Data_Centers_Through_Adiabatic_Reversible_CMOS. [Accessed 05-06-2026].
- [27] Andrew N. Sloss. 2022. *Evolution of the Semiconductor Industry, and the Start of X Law*. Springer Nature Singapore, Singapore, 197–209. doi:10.1007/978-981-16-

- [28] Andrew N. Sloss and John Goodacre. 2005. Parallelism and the ARM instruction set architecture. *Computer* 38, 7 (2005), 42–50.
- [29] Shuang Wang, Yao Zhang, Yiqiang Liu, Yufeng Li, and Xiang Li. 2018. 3D Integrated Circuit Cooling with Microfluidics. *Micromachines* 9, 6 (2018). doi:10.3390/mi9060287
- [30] Sally Ward-Foxton. [n. d.]. TSMC Defends Transistor Scaling Amid Huawei's 'Her's Law' Proposal. <https://www.eetimes.com/tsmc-defends-transistor-scaling-amid-huaweis-hers-law-proposal/>. [Accessed 04-06-2026].
- [31] Sally Ward-Foxton. 2026. Necessity is the Mother of Invention: Huawei Replaces Moore's Law With Her's Law. <https://www.eetimes.com/necessity-is-the-mother-of-invention-huawei-replaces-moores-law-with-hers-law/>. [Accessed 04-06-2026].

A Broad economic framing

Here, we present a framing of compute efficiency in broad economic terms and discuss how it relates to device-level characteristics.

A.1 General model

In general, we can speak of the efficiency of any system or process that converts inputs (which could be anything) to outputs (which could also be anything, not necessarily the same type of thing) as follows.

First, some definitions:

- Let P be the amount of some valued *product produced* by a given instance of the system or process over a specified time interval, expressed in suitable units;
- Let C be the amount of some valued *cost consumed* by the system or process in that time interval, in the service of producing P , and expressed in suitable units (which, in general, need not be the same units as those for P). This cost need not be strictly financial in nature; for example, it could instead be quantified in terms of energy consumed or any consumable resource.

For simplicity, we can imagine that C is the *only* consumable input to the process, or, if there are a variety of different kinds of inputs, that all such supplies are expressed in terms of compatible units—such as dollar cost or energy consumed in the supply chain while producing those inputs.

Now, in this general setup, a generic characterization of efficiency η is simply:

$$\eta = \frac{P}{C}. \quad (4)$$

A.2 Application to computing

In the context of a computer system, we can choose the product P and cost C (per the above framing) as follows:

- Here we can assign P to be the *amount of useful computational work performed* by the system over the given period, which may appropriately be chosen to be the machine's operational lifetime. The amount of work can be characterized in terms of the number n_{op} of useful operations (of some type) performed by the system. For example, number of floating-point operations (of a given type). Or, if a variety of different types of operations are performed, they could be converted to an equivalent number of standardized operations.
- And, for economic purposes, C could be defined as the *total lifetime cost of ownership* (TCO) of the system, including

manufacturing, deployment, and disposal costs, as well as the cost of operating expenses (including energy) over the system's lifetime. Grouping things together into just non-recurring expenses (manufacturing, deployment, disposal) plus operating expenses gives

$$C = C_{\text{tot}} = C_{\text{nre}} + C_{\text{oper}}. \quad (4)$$

We can then say that $\eta = \eta_{\text{cec}} = P/C = n_{\text{op}}/C_{\text{tot}}$ expresses the overall *cost-efficiency of computing*.

If we wish, we can also express P and C in annualized terms by amortizing the totals over the machine's operational lifetime. More sophisticated cost models that account for e.g. the discounted net present value of future returns on expended funds, equipment depreciation curves, and so forth are not considered here.

Many of the operating expenses associated with running a large computing system can be considered *energy-proportional costs* and expressed as:

$$C_{\text{oper}} = c_{\text{en}} \cdot E_{\text{oper}}, \quad (5)$$

where c_{en} is a *cost coefficient* (cost per unit of energy supplied), and E_{oper} is the energy supplied to the system over the given period.

If we consider the energy supplied per operation performed E_{op} , and a corresponding time interval t_d (a delay value corresponding to the initiation interval for each functional unit performing operations), we can divide through the efficiency formula by $P = n_{\text{cyc}} \cdot n_{\text{dev}}$, where n_{cyc} is the number of operational cycles (each of duration t_d) carried out over the machine's lifetime, and n_{dev} is the number of parallel devices (in the sense of functional units) within the machine.⁷ Then, the efficiency expression $\eta = P/C$ reduces to

$$\eta_{\text{cec}} = \frac{1}{c_{\text{en}} \cdot E_{\text{op}} + c_{\text{dev},t} \cdot t_d}. \quad (6)$$

where $c_{\text{dev},t}$ is the amortized NRE cost per device per unit of time. The denominator in this expression represents the system's TCO per device-cycle.

Let us briefly explain why this is so in more detail. The total number of operations performed over the system's lifetime is $P = n_{\text{op}} = n_{\text{cyc}} \cdot n_{\text{dev}}$, assuming full utilization of all devices, and if we divide P in the numerator by this we get 1. Meanwhile, the total cost of energy supplied is

$$C_{\text{oper}} = c_{\text{en}} \cdot E_{\text{oper}} \quad (7)$$

$$= c_{\text{en}} \cdot (n_{\text{op}} \cdot E_{\text{op}}), \quad (8)$$

and dividing this by $P = n_{\text{op}}$ gives just $c_{\text{en}} \cdot E_{\text{op}}$, the left-hand subexpression in the denominator. Meanwhile, the total NRE can be expressed in terms of an amortized per-device cost time the number of devices,

$$C_{\text{nre}} = c_{\text{dev}} \cdot n_{\text{dev}}, \quad (9)$$

and we can further break down c_{dev} into the amortized device cost per unit time times the system lifetime $L = n_{\text{cyc}} \cdot t_d$,

$$c_{\text{dev}} = c_{\text{dev},t} \cdot L \quad (10)$$

$$= c_{\text{dev},t} \cdot (n_{\text{cyc}} \cdot t_d), \quad (11)$$

so we have

$$C_{\text{nre}} = [c_{\text{dev},t} \cdot (n_{\text{cyc}} \cdot t_d)] \cdot n_{\text{dev}}, \quad (12)$$

and when we divide this through by $n_{\text{op}} = n_{\text{cyc}} \cdot n_{\text{dev}}$, we get $c_{\text{dev},t} \cdot t_d$, the amortized manufacturing cost per device-cycle, which is the

⁷ Assuming full utilization.

right-hand subexpression in the denominator. Thus we recover eq. 6.

A few important general observations to note, based on eq. 6:

- (1) There will be *diminishing returns* (in terms of efficiency gains) from reducing either of the terms in the denominator of eq. 6 (either the energy-related term, or the term related to device speed and cost) in isolation—because the other term will just dominate the total cost.
- (2) However, if hypothetically *both* the $c_{\text{en}} \cdot E_{\text{op}}$ term *and* the $c_{\text{dev},t} \cdot t_d$ term were respectively reduced by a factor of $N\times$ each, the overall cost would be reduced by $N\times$, and overall efficiency η would increase by a factor of $N\times$.
- (3) Slightly more specifically, one way to reduce both terms would be to reduce both E_{op} (energy supplied per operation) and t_d (operation delay) by a factor of N , which would reduce the *energy-delay product* $E_{\text{op}}t_d$ (a popular figure of merit) by N^2 .
- (4) However, reducing the energy-delay product is *not* the *only* way to improve overall cost-efficiency. Rather than reducing energy supplied, we could instead reduce the cost of energy. And rather than reducing delay per operation cycle, we could instead reduce the manufacturing cost per device or the operational lifetime of the system. Or some combination of all these factors.
- (5) As another example, suppose that the energy-delay product (a.k.a. energy coefficient) $c_E = E_{\text{op}}t_d$ is fixed, so that reducing the net energy supplied per operation E_{op} implies increasing the operation delay t_d . (This is the case, for example, when doing adiabatic reversible computing (ARC) using a given device technology.) The overall cost efficiency of computing η_{cec} can nevertheless *still improve*, as long as $c_{\text{dev},t}$ (amortized NRE per device per unit time) remains (or becomes) small enough so that the total cost per operation $C_{\text{op}} = c_{\text{en}} \cdot E_{\text{op}} + c_{\text{dev},t} \cdot t_d$ (the denominator of eq. 6) is still less than before even though t_d is greater. So, for example, the overall cost efficiency of ARC-based systems can continue improving as manufacturing costs decline or system lifetimes increase, even if device-level energy-delay products hit a floor.

The above provides a very general framework for characterizing the cost efficiency of computing.

A.3 Refined model

In this section, we dig a little bit deeper into the above line of thought, refining the efficiency model a bit to explore the implications in scenarios where increases in energy efficiency trade off against not just the speed but also the physical size (in terms of *e.g.* area, volume, mass) of individual computing devices (logic gates or functional units).

Again, as in the previous section, we start by considering the overall cost-efficiency of computing η_{cec} , defined as

$$\eta_{\text{cec}} = \frac{n_{\text{op}}}{C_{\text{tot}}} = \frac{1}{C_{\text{op}}}, \quad (13)$$

where $C_{\text{tot}} = C_{\text{nre}} + C_{\text{oper}}$ includes both non-recurring (*e.g.* manufacturing) costs and operating costs (mainly energy), and $C_{\text{op}} = C_{\text{tot}}/n_{\text{op}}$ is the amortized *total cost per operation*.

So far, this is the same as discussed previously, except now, we will break down C_{op} in slightly more detail, writing it as:

$$C_{\text{op}} = c_{\text{en}} \cdot E_{\text{op}} + c_M(s_{\text{dev}} \cdot t_d), \quad (14)$$

which we explain as follows.

As before, c_{en} is the cost coefficient for operating costs proportioned to energy use, which can be quantified as the total operating costs C_{oper} ascribable to supplying power and cooling to the computing system over its operational lifetime, divided by the total energy delivered E_{oper} :

$$c_{\text{en}} = \frac{C_{\text{oper}}}{E_{\text{oper}}} = \frac{C_{\text{oper}}}{p_{\text{avg}} \cdot L}, \quad (15)$$

where in the last form we have expressed E_{oper} in terms of the system's average total instantaneous net power draw p_{avg} times its operational lifetime L .

As before, E_{op} is the net energy supplied, dissipated, and removed (as heat) per computational operation performed, so

$$E_{\text{op}} = \frac{E_{\text{oper}}}{n_{\text{op}}} = \frac{p_{\text{avg}} \cdot L}{n_{\text{op}}}. \quad (16)$$

The new quantity s_{dev} represents the average physical size (in, say, volume or mass units, whichever we prefer to deal with) associated with individual devices or computing elements within the machines. We could quantify this by dividing the total size of the system S_{sys} by the number of devices n_{dev} within it:

$$s_{\text{dev}} = \frac{S_{\text{sys}}}{n_{\text{dev}}}. \quad (17)$$

And finally, c_M is a comprehensive *manufacturing cost coefficient*, obtained by taking the total NRE associated with manufacturing (and deploying, and disposing of) the whole computing system, and amortizing it over the number of devices, and their size, and the system lifetime L :

$$c_M = \frac{C_{\text{nre}}}{S_{\text{sys}} \cdot L} = \frac{C_{\text{nre}}}{n_{\text{dev}} \cdot s_{\text{dev}} \cdot L}. \quad (18)$$

The rationale for all this is that now, the second term in the refined cost-per-operation equation, eq. 14, is the amortized manufacturing cost per operation (or device-cycle), the same as it was before in eq. 6, but now with the device size s_{dev} explicitly accounted for as a separate factor.

The purpose, now, of all of this, is just to allow us to note, with reference to eq. 14, that the total cost per operation C_{op} can still decrease (and thus the overall cost efficiency of computing η_{cec} can still increase) even if reducing energy consumed per operation E_{op} requires increasing *both* the per-device size s_{dev} and device-cycle delay t_d , so long as the comprehensive manufacturing cost coefficient c_M remains small enough, or decreases enough.

The implication is that the overall cost-efficiency of reversible computers can continue increasing *even* if the trend on that scaling path is one in which individual devices become larger and slower as their energy efficiency improves. An example of a sufficient condition for this is if the deployed lifetime L of systems continues increasing.

B Efficiency equation

Total power dissipation (P_{total}) in semiconductor circuits (primarily CMOS) is the sum of dynamic and static power, as shown in Equation 1.⁸

$$P_{total} = P_{dynamic} + P_{static} \quad (19)$$

This breaks down into more detail, as shown in Equation 2.

$$P_{total} = \alpha \cdot C \cdot V_{DD}^2 \cdot f + V_{DD} \cdot I_{leak} \quad (20)$$

- P_{total} : Total power consumption of the logic circuit
- α : Switching activity factor (the % of gates actively changing states)
- C : Total physical and parasitic capacitance of the circuit
- V_{DD} : Supply voltage applied to the chip
- f : Clock frequency (speed of the processor)
- I_{leak} : Leakage current flowing through the idle transistors

Equation 2 shows the importance of frequency in power draw. The general rule is that the higher the frequency, the greater the power drawn when carrying out computations.

B.1 Efficiency

Efficiency is a dimensionless ratio—the units cancel. In other words, the units cancel themselves out: the watts cancel for η_p ; the joules cancel for η_e .

POWER EFFICIENCY: η_p is evaluated at an instant in time or at a specified operating point; see Equation 3. The ratio of output power to input power at a moment. Useful for identifying peaks and troughs of drawn computational data.

$$\eta_p = \frac{P_{out}}{P_{in}}. \quad (21)$$

ENERGY EFFICIENCY: η_e is evaluated over an interval of time Δt ; see Equation 4. The ratio of total energy out to total energy in across that interval of time. Useful for identifying the costs of completing a workload (computation over time).

$$\eta_e = \frac{E_{out}}{E_{in}} = \frac{\int_{t_0}^{t_0+\Delta t} P_{out}(t) dt}{\int_{t_0}^{t_0+\Delta t} P_{in}(t) dt}. \quad (22)$$

Both η_p and η_e are dimensionless; the difference is that η_e requires you to specify a time window. Time is a *parameter of the definition*, not a *dimension of the result*. Two consequences:

- If P_{in} and P_{out} are constant over Δt , then $\eta_e = \eta_p$ and the choice of interval are immaterial.

⁸Where traditional CMOS uses paired transistors, the circuit only uses power when switching, making it fast and energy-efficient. CPUs and GPUs are all implemented using CMOS technology.

- If they vary, which is more likely, η_e depends on the interval, and in general $\eta_e \neq \eta_p$ because the integral of a ratio is not the ratio of integrals.

This is where the distinction stops being academic and starts driving billion-dollar architectural decisions. Semiconductors are unusual because power and energy can diverge dramatically, and the design optimum depends entirely on which one you're targeting.

USEFUL WORK: Where energy efficiency becomes important is when it is measured by the amount of *useful workload* completed for a given amount of energy. Peak power or peak performance is not enough to measure because a system may look efficient at one point while wasting energy across the full duty cycle. This means it is better to ask *how much useful work is produced per joule consumed?*

$$\eta_{energy} = \frac{W_{useful}}{E_{total}} \quad (23)$$

where:

- η_{energy} is energy efficiency
- W_{useful} is the useful workload completed
- E_{total} is the total energy consumed

With energy efficiency being power integrated over time, we can simplify the equation to:

$$E_{total} \approx P_{active} t_{active} + P_{idle} t_{idle}. \quad (24)$$

Looking at useful work done is not just about reducing instantaneous power, but about maximizing the amount of valuable computation completed per unit of total energy consumed. We see that in the units used as measuring metrics, i.e., *operations-per-joule*, *inference-per-watt*, or *tokens-per-watt*. **How do we achieve more useful work?**

C Reversible computing and efficiency

At Vaire Computing, we study how the techniques of adiabatic and reversible CMOS (what we call ARC) can be used to improve the energy efficiency of computing, which in turn can improve the overall cost-efficiency of computing by the arguments of Appendix A.

Framed in terms of *useful work* as discussed in Appendix B, let us again use W_{useful} to denote *the amount of useful computational work* carried out in the service of some application(s) of interest. This can be quantified in terms of the number n_{op} of useful *operations performed* of some relevant type (this could, for example, count integer or floating point operations of a given precision and format). So, we can write just $W_{useful} \equiv n_{op}$ as a matter of definition.

Meanwhile, let us again let E_{total} represent the total energy consumed (and dissipated!) by the system over the course of the computation. So now we can focus on *computational energy efficiency* η_{ce} as follows:

$$\eta_{ce} = \frac{W_{useful}}{E_{total}}. \quad (25)$$

Note this efficiency concept can be understood as a special case of the general efficiency model of eq. 3 in Appendix A: The *valued product produced* is exactly the useful computational work performed, while the *valued cost consumed* is exactly the total energy dissipated E_{total} . (Note that in this context, we are temporarily ignoring non-recurring manufacturing/deployment/disposal costs of the system, and just focusing on energy use.)

Now, in the context of ARC, eq. 19 needs to be reinterpreted, in the service of drawing an important distinction: Namely, the distinction between the amount of power delivered (a.k.a. transferred, supplied) to a given system, versus the amount of power consumed (a.k.a. dissipated to heat) by the system. These two quantities are not necessarily always the same, in general! Because in general, (electrical) power can flow both to and from a given system (as it does in AC circuits). Power that's been returned can, in general, be recovered and later reused—such as in an electric vehicle that recharges its battery when braking or traveling downhill. And, in between being delivered and being recovered, energy can be stored temporarily within the system, such as in a flywheel, or in a vehicle's kinetic or potential energy.

We can therefore discuss a quantity which we can call the *energy-recovery efficiency*:

$$\eta_{\text{er}} = \frac{E_r}{E_d} \quad (26)$$

where E_d is energy delivered and E_r is energy recovered. (Structurally, this is the same as eq. 22; one could say that here, we are just changing the names to protect the innocent—or really, for clarity: $E_{\text{in}} = E_d$ is energy delivered “in,” from the power supply to the computing system; while $E_{\text{out}} = E_r$ is energy recovered from the computing system and sent back “out” to the power supply.)

In conventional CMOS digital designs, however, fully 100% of all the electrical power P_{total} that's delivered to a given digital circuit or subsystem ends up getting dissipated to heat. Absolutely zero energy is recovered, so these systems are 0% efficient in that sense. For these systems, $\eta_{\text{er}} = 0$ always.

However, in ARC designs, the situation is different: Although the static power P_{static} is still all dissipated, the dynamic power P_{dynamic} is not—a substantial fraction of it is reused and recovered. Focusing now just on the recovery of dynamic power, we can write

$$\eta_{\text{dr}} = \frac{P_{\text{dyn,rec}}}{P_{\text{dynamic}}} \quad (27)$$

where $P_{\text{dyn,rec}}$ is the average dynamic power recovered. In situations where we can neglect static power and dynamic power is roughly constant (such as a system with high utilization operating near its peak rated performance), we can say $\eta_{\text{er}} \approx \eta_{\text{dr}}$, i.e., the total energy recovery efficiency is about the same as the fraction of dynamic power recovered.

In such cases, we also define a number we call the *R-factor* or *energy recovery factor* R_{fac} , which expresses the ratio between the total dynamic power (or energy) delivered and the portion of it that is dissipated (as opposed to being recovered):

$$R_{\text{fac}} = \frac{1}{1 - \eta_{\text{er}}}, \quad (28)$$

or we can solve for η_{er} and write:

$$\eta_{\text{er}} = 1 - \frac{1}{R_{\text{fac}}}. \quad (29)$$

In conventional CMOS, since no energy is recovered, $R_{\text{fac}} = 1$. In ARC, a goal is to have $R_{\text{fac}} \gg 1$, and if all else were equal, the value of R_{fac} would also correspond to the overall improvement in computational energy efficiency η_{ce} .

However, all else is not equal. In general, ARC designs incur some overheads in terms of added complexity at the level of logic circuit architecture. These tend to increase the total amount C of capacitance that needs to be switched in order to carry out a given computational operation, and thus the dynamic power P_{dynamic} that must be delivered to achieve a given level of computational throughput is increased. However, despite this, ARC designs can still increase overall energy efficiency. Let us see how.

Let us define the *X-factor* or *chi-factor* χ_{fac} to be the ratio of dynamic power that must be delivered, in the ARC versus conventional CMOS cases, to achieve a given level of throughput (performance):

$$\chi_{\text{fac}} = \frac{P_{\text{dyn,arc}}}{P_{\text{dyn,conv}}}. \quad (30)$$

However, only a fraction $1/R_{\text{fac}}$ of the dynamic power in the ARC design is actually dissipated, so, the total dynamic power dissipation in the ARC design is actually only

$$P_{\text{diss,arc}} = \frac{\chi_{\text{fac}}}{R_{\text{fac}}} P_{\text{dyn,conv}}. \quad (31)$$

We can thus define the following quantity to express the ratio between the power dissipation in the ARC versus conventional cases to achieve a given throughput:

$$V_{\text{fac}} = \frac{P_{\text{dyn,conv}}}{P_{\text{diss,arc}}} = \frac{R_{\text{fac}}}{\chi_{\text{fac}}}. \quad (32)$$

This is also the ratio between computational energy efficiency (useful computational work per unit energy dissipated) in the ARC versus conventional cases:

$$\frac{\eta_{\text{ce,arc}}}{\eta_{\text{ce,conv}}} = V_{\text{fac}} = \frac{R_{\text{fac}}}{\chi_{\text{fac}}}. \quad (33)$$

So, adiabatic reversible designs can increase the amount of useful computational work that get accomplished per unit of energy that is consumed/dissipated, so long as $R_{\text{fac}} > \chi_{\text{fac}}$. And thus, by the arguments of Appendix A, they can also increase the overall cost-efficiency of computing, despite their complexity overheads.